第四章 流程控制与模式匹配

4.1 if else

- 和C语言语法类似，但Rust中的if else是表达式而不是语句，可以用来赋值
- 语法：

1. 基本的 if

```rust
if condition {
    // 代码块
}
```

2. if-else

```rust
if condition {
    // 条件为真时执行
} else {
    // 条件为假时执行
}
```

3. if-else if-else

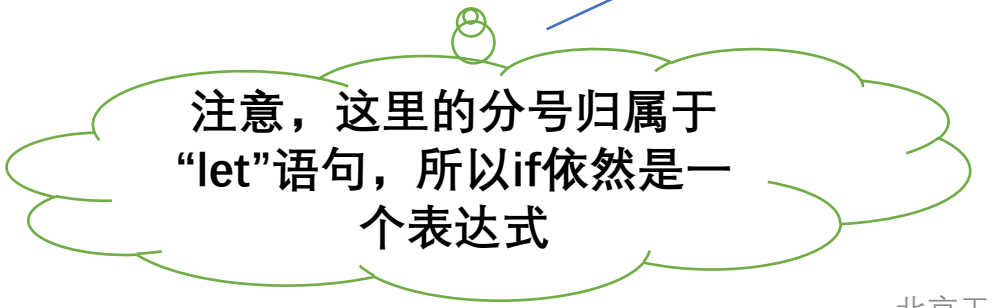
```rust
if condition1 {
    // 条件1为真时执行
} else if condition2 {
    // 条件2为真时执行
} else {
    // 所有条件都为假时执行
}
```

4.1 if else

- 作为表达式使用：

基本用法

```rust
let number = if condition { 5 } else { 6 };
println!("The number is: {}", number); // 输出 5 或 6
```



多分支返回值

```rust
let score = 85;
let grade = if score >= 90 {
    "A"
} else if score >= 80 {
    "B"
} else if score >= 70 {
    "C"
} else {
    "F"
};
println!("Grade: {}", grade); // 输出 "B"
```

4.1 if else作为表达式时的规则

1. 所有分支必须返回相同类型

```rust
// ✅ 正确 - 所有分支都返回 i32
let x = if true { 1 } else { 2 };

// ❌ 错误 - 类型不匹配
let y = if true { 1 } else { "two" }; // 编译错误!
```

2. 分支必须是表达式，不能是语句

```rust
// ✅ 正确 - 表达式
let a = if condition { 5 } else { 6 };

// ❌ 错误 - 分支是语句（有分号）
let b = if condition { 5; } else { 6; }; // 编译错误!
```

3. else 分支可选，但作为表达式时必须要有

```rust
// 作为语句时，else 可选
if condition {
    println!("It's true!");
}
// 这里没有返回值

// 作为表达式时，必须要有 else
let result = if condition {
    "true"
} else {
    "false"
};
```

4.2 循环： for、while 和 loop

- **for循环主要用于迭代集合中的元素**

1. 迭代范围

```rust
// 迭代 0 到 4（不包含 5）
for i in 0..5 {
    println!("i = {}", i);
}

// 迭代 0 到 5（包含 5）
for i in 0..=5 {
    println!("i = {}", i);
}
```

2. 迭代一个“集合”，比如数组、字符串等

```rust
let arr = [1, 2, 3, 4, 5];

// 直接迭代值
for num in arr {
    println!("数字: {}", num);
}

// 迭代引用（避免所有权转移）
for num in &arr {
    println!("数字引用: {}", num);
}
```

- for循环涉及到复杂变量所有权转移（数组例外，它是“复制”），通常使用引用形式

```rust
for item in &container {
    // ...
}
```

```rust
for item in &mut collection {
    // ...
}
```

- continue、break命令

4.2 循环：for、while 和 loop

- while和C语言差不多
- loop是一个死循环，需要搭配break使用

循环类型	适用场景	示例
`while`	条件循环	`while x < 10 { ... }`
`while let`	模式匹配循环	`while let Some(x) = stack.pop() { ... }`
`loop`	无限循环	`loop { ... }`
`for`	集合遍历	`for x in vec { ... }`

4.3 模式匹配match（一种高级switch-case）

- 回忆C语言的switch-case语法结构：

```c
switch (expression) {
    case constant1:
        // 代码块1
        break;
    case constant2:
        // 代码块2
        break;
    case constant3:
        // 代码块3
        break;
    default:
        // 默认代码块
        break;
}
```

- 在Rust中是match

```
match target {
    模式1 => 表达式1,
    模式2 => {
        语句1;
        语句2;
        表达式2
    },
    _ => 表达式3
}
```

4.3 模式匹配match——基本语法

匹配整数值

```rust
fn main() {
    let number = 3;

    match number {
        1 => println!("一"),
        2 => println!("二"),
        3 => println!("三"),
        _ => println!("其他数字"),
    }
}
```

匹配枚举

```rust
enum Color {
    Red,
    Green,
    Blue,
}

fn main() {
    let color = Color::Green;

    match color {
        Color::Red => println!("红色"),
        Color::Green => println!("绿色"),
        Color::Blue => println!("蓝色"),
    }
}
```

- 要穷举出所有可能，不能漏，可以用 _ 来代表未列出的所有可能性
- 每一个分支都必须是一个表达式，返回值的类型必须相同
  - 注意这里的println!(“一”)，返回了什么？
- match本身也是一个表达式，和if-else类似

4.3 模式匹配match——基本语法

- 模式即**数据的形状、值或结构**

1. 字面量模式

匹配具体的值

```rust
let x = 1;
match x {
    1 => println("一"),      // 模式：字面量1
    2 => println!("二"),     // 模式：字面量2
    _ => println!("其他"),   // 模式：通配符
}
```

2. 变量模式

绑定值到变量

```rust
let x = 5;
match x {
    num => println!("值是: {}", num),
}
```

3. 通配符模式

匹配任何值

```rust
let x = 42;
match x {
    _ => println!("匹配任何值"),
}
```

4. 或模式

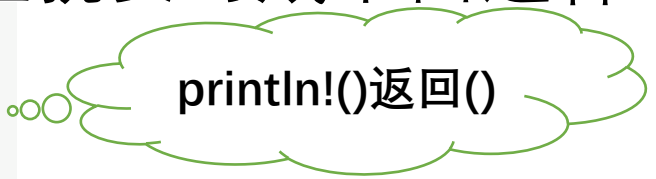
匹配多个模式之一

```rust
let x = 2;
match x {
    1 | 3 | 5 | 7 | 9 => println!("奇数"),    // 模式：1或3或5...
    2 | 4 | 6 | 8 | 10 => println!("偶数"),   // 模式：2或4或6...
    _ => println!("其他"),
}
```

4.3 模式匹配if-let——基本语法

- 有时会遇到只有一个模式的值需要被处理，其它值直接忽略的场景，如果用 match 来处理就要写成下面这样：

```
let v = Some(3u8);
match v {
    Some(3) => println!("three"),
    _ => (),
}
```



- 如果只想要对 Some(3) 模式进行匹配，不想处理任何其 Some<u8> 值或 None 值，可以用if let语句

4.3 模式匹配if-let——基本语法

- 普通用法：

```
let some_value = Some(5);

if let Some(x) = some_value {
    println!("找到值: {}", x);
    // 其他处理逻辑...
}
// 如果不匹配，什么都不做
```

- 结合else用法

```
fn main() {
    let some_value = Some(5);

    if let Some(x) = some_value {
        println!("找到值: {}", x);
    } else {
        println!("没有值");
    }

    let none_value: Option<i32> = None;
    if let Some(x) = none_value {
        println!("找到值: {}", x);
    } else {
        println!("没有值"); // 输出: 没有值
    }
}
```

4.3 模式匹配while-let——基本语法

- 一个与 if let 类似的结构是 while let 条件循环，它允许只要模式匹配就一直进行 while 循环

```
// Vec是动态数组
let mut stack = Vec::new();

// 向数组尾部插入元素
stack.push(1);
stack.push(2);
stack.push(3);

// stack.pop从数组尾部弹出元素
while let Some(top) = stack.pop() {
    println!("{}", top);
}
```



4.3 模式匹配——基本特征与概念

- 模式绑定
- Option<T>
- 变量遮蔽

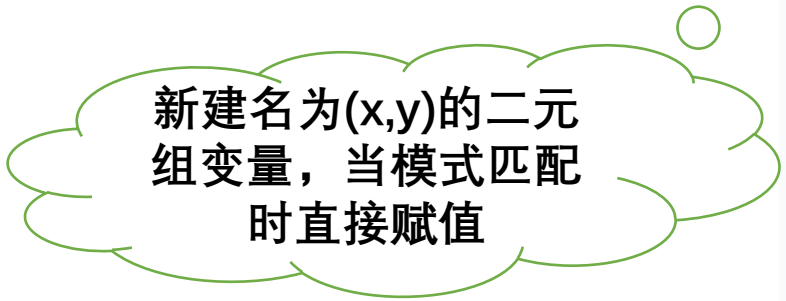
4.3 模式匹配—模式绑定

- 模式绑定：**匹配时提取，**模式绑定发生在模式匹配的过程中，**在匹配成功的同时提取值**

1.尝试匹配 - 检查数据是否符合模式结构

2.绑定变量 - 匹配则提取相应部分并绑定到变量

3.执行代码 - 在分支中使用绑定变量




```
let point = (10, 20);

match point {
    (x, y) => {
        // 1. 检查：point 是二元组 ✓
        // 2. 绑定：第一个元素绑定到 x，第二个绑定到 y
        // 3. 执行：使用 x 和 y
        println!("x={}, y={}", x, y);
    }
}
```

4.3 模式匹配—各种场景中的模式绑定

场景1：枚举解构

```rust
enum Message {
    Move { x: i32, y: i32 },
    Write(String),
}

let msg = Message::Move { x: 10, y: 20 };

match msg {
    Message::Move { x, y } => {
        // 绑定结构体字段到变量 x, y
        println移动到 ({}, {})", x, y);
    }
    Message::Write(text) => {
        // 绑定 String 到变量 text
        println!("写入: {}", text);
    }
}
```

场景2：带条件的绑定（模式守卫）

```rust
let some_value = Some(50);

match some_value {
    Some(x) if x > 0 && x < 100 => {
        // 匹配 Some，且值在 0-100 之间，绑定到 x
        println!("有效值: {}", x);
    }
    Some(x) => {
        // 匹配 Some，但值不在范围内，也绑定到 x
        println!("超出范围: {}", x);
    }
    None => {}
}
```

4.3 模式匹配—各种场景中的模式绑定

场景3：多层嵌套绑定

```rust
let complex = Some(Some(Some(42)));

match complex {
    Some(Some(Some(inner_value))) => {
        // 穿透三层 Some，绑定最内层值到 inner_value
        println!("最内层值: {}", inner_value);
    }
    _ => println!("结构不匹配"),
}
```

场景4：部分绑定

```rust
let triple = (1, 2, 3);

match triple {
    (first, _, third) => {  // 只绑定第一个和第三个元素
        println!("第一个: {}, 第三个: {}", first, third);
    }
}
```

4.3 模式匹配—模式绑定的作用域

- 1、模式绑定只在**当前匹配分支中有效**（理解为局部变量）：

```rust
let value = Some("hello".to_string());

match value {
    Some(s) => {  // s 绑定到 String
        println!("在分支内: {}", s);  // ✅ 可以访问 s
    }
    None => {}
}

// println!("在分支外: {}", s);  // ❌ 错误: s 不在作用域内
```

- 2、涉及到复杂变量则有移动（所有权转移）

4.3 模式匹配——Option<T>

- 在枚举中介绍过Option枚举（包含有值Some(T), 空： None）
- 其他语言中“NULL”是**值**； Rust中None被包在了Option枚举中，从值变成了一种**变量类型**
  - 编译器无法检查值，但可以检查变量类型，避免出现操作空指针导致系统运行时崩溃。
  - 程序员需要在包含空值的时候使用Option枚举，而不是默认将空值看成是普通值的一个特例。

4.3 模式匹配——Option<T>

- 逻辑含义是“一个变量可能有值也可能为空”—使用match来处理有值和空的情况

```
fn plus_one(x: Option<i32>) -> Option<i32> {
    match x {
        None => None,
        Some(i) => Some(i + 1),
    }
}

let five = Some(5);
let six = plus_one(five);
let none = plus_one(None);
```

1、x是输入参数
2、i为模式绑定的变量
3、match要求分支返回类型一致，None、Some(i)都是Option枚举类型变量

4.3 模式匹配——Option<T>

- 模式绑定可以解包Option，把Some包的变量解出来

```
let value = Some(42);

match value {
    //  ↓ 模式部分      ↓ 绑定变量
    Some(x) => { // 当匹配 Some 时，将其内部值绑定到 x
        println!("值是: {}", x); // x = 42
    }
    None => {}
}
```

新建名为x的变量，将
Some(42)包裹的42这个值
解包到x变量中

4.3 模式匹配——变量遮蔽问题

- 当在 match 分支中使用与外部变量同名的绑定时，会发生变量遮蔽：

```
fn main() {
    let x = 10;

    let value = Some(5);

    match value {
        Some(x) => {  // 这里遮蔽了外部的 x
            println!("内部 x: {}", x);  // 输出 5
        },
        None => (),
    }

    println!("外部 x: {}", x);  // 输出 10，遮蔽结束后恢复
}
```

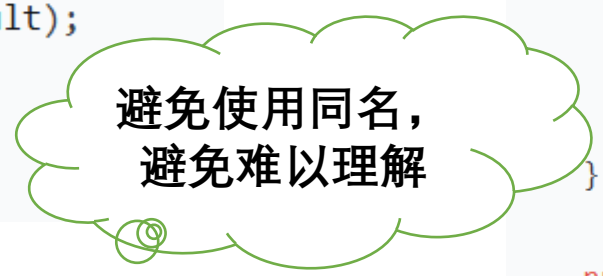


4.3 模式匹配——变量遮蔽（常见情况）

- 意外遮蔽

```rust
fn process_data(data: Option<String>) {
    let default = "default".to_string();

    match data {
        Some(default) => {  // 意外遮蔽了外部的 default
            println!("处理数据: {}", default);
            // 这里无法访问外部的 default 变量
        },
        None => {
            println!("使用默认值: {}", default);
        }
    }
}
```



- 在循环中遮蔽

```rust
fn main() {
    let mut counter = 0;
    let values = vec![Some(1), None, Some(2)];

    for value in values {
        match value {
            Some(counter) => {  // 遮蔽了外部的 counter
                println!("找到值: {}", counter);
                // counter += 1;  // 这不会影响外部的 counter
            },
            None => {
                counter += 1;  // 这里访问的是外部的 counter
            }
        }
    }

    println!("最终计数: {}", counter);
}
```

4.3 模式匹配——案例

- 模式绑定：方便的处理数据的解构和提取
- 看一个C语言的例子：网络数据包的处理，网络里面有多种数据包，每种由不同的结构体定义并包含不同的数据类型
  - 需求：写一个函数提取多种数据结构中的内容

4.3 模式匹配

- C语言需要手动解析复杂结构的数据

```c
// 定义数据包类型
typedef enum {
    DATA_PACKET,
    CONTROL_PACKET,
    ERROR_PACKET
} PacketType;

// 定义数据包结构
typedef struct {
    PacketType type;
    union {
        struct {
            int sequence;
            char* data;
        } data_packet;
        struct {
            int command;
            int parameter;
        } control_packet;
        struct {
            int error_code;
            char* message;
        } error_packet;
    } content;
} Packet;
```

```c
// 处理数据包的函数
void process_packet(Packet* packet) {
    switch (packet->type) {
        case DATA_PACKET: {
            // 需要手动提取每个字段
            int seq = packet->content.data_packet.sequence;
            char* data = packet->content.data_packet.data;
            printf("数据包: 序列号=%d, 数据=%s\n", seq, data);
            break;
        }
        case CONTROL_PACKET: {
            // 同样需要手动提取
            int cmd = packet->content.control_packet.command;
            int param = packet->content.control_packet.parameter;
            printf("控制包: 命令=%d, 参数=%d\n", cmd, param);
            break;
        }
```

```c
int main() {
    // 创建一个数据包
    Packet data_pkt = {
        .type = DATA_PACKET,
        .content.data_packet = {
            .sequence = 1001,
            .data = "Hello World"
        }
    };

    process_packet(&data_pkt);
    return 0;
}
```

4.3 模式匹配

- Rust利用模式匹配自动解构

1. 简洁性

- C: 需要多行代码手动提取每个字段
- Rust: 一行模式绑定自动提取所有字段

2. 安全性

- C: 可能忘记提取某些字段或提取错误
- Rust: 编译器保证模式匹配的完整性

3. 可读性

- C: 提取逻辑和业务逻辑混合
- Rust: 数据结构形状清晰可见

4. 重构友好

如果添加新字段:

- C: 需要修改每个switch case
- Rust: 编译器会提示需要更新所有匹配分支

```rust
// 定义数据包枚举
enum Packet {
    Data { sequence: i32, data: String },
    Control { command: i32, parameter: i32 },
    Error { code: i32, message: String },
}

fn process_packet(packet: Packet) {
    match packet {
        // 模式绑定：自动提取并绑定字段到变量
        Packet::Data { sequence, data } => {
            println!("数据包: 序列号={}, 数据={}", sequence, data);
        }
        // 模式绑定：自动提取命令和参数
        Packet::Control { command, parameter } => {
            println!("控制包: 命令={}, 参数={}", command, parameter);
        }
        // 模式绑定：自动提取错误代码和消息
        Packet::Error { code, message } => {
            println!("错误包: 代码={}, 消息={}", code, message);
        }
    }
}
```

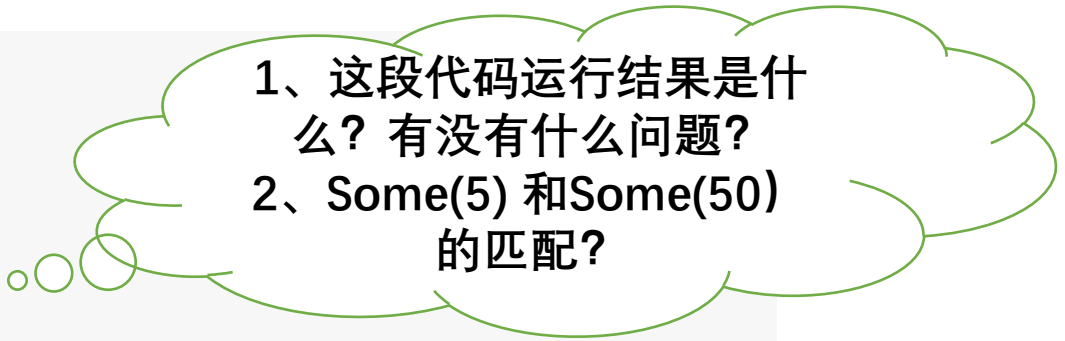
4.3 模式匹配——案例

- 匹配命名变量

```
fn main() {
    let x = Some(5);
    let y = 10;

    match x {
        Some(50) => println!("Got 50"),
        Some(y) => println!("Matched, y = {:?}", y),
        _ => println!("Default case, x = {:?}", x),
    }

    println!("at the end: x = {:?}, y = {:?}", x, y);
}
```



4.3 模式匹配——案例

- Some(5) 和 Some(50) 都是 Option<i32> 类型，但模式匹配检查的是具体的值，而不是类型。模式匹配在检查完枚举变体后，还会进一步检查内部的值是否匹配。
- 如果只想匹配任意Some，不关心内部具体值
  - 使用变量绑定
  - 使用通配符“_”，即default
  - 使用if条件（p15的“模式守卫”）

4.3 模式匹配——模式适用场景

- match
- if let
- while let
- for循环

```
for i in 0..5 {
    println!("{}", i);
}
```

- let语句：匹配的值绑定到变量上，变量名也是一种模式

```
let (x, y, z) = (1, 2, 3);（正确）
let (x, y) = (1, 2, 3);（错误）
```

- 函数参数：参数就是一种模式

```
fn foo(x: i32) {
    // 代码
}
```

- let 和 if let

```
let Some(x) = some_option_value;
```

  - 这段代码是错误的，左右两边类型不匹配（右边有可能存在None）
  - 类似 let , for和match 都必须要求完全覆盖匹配，才能通过编译(不可驳模式匹配)